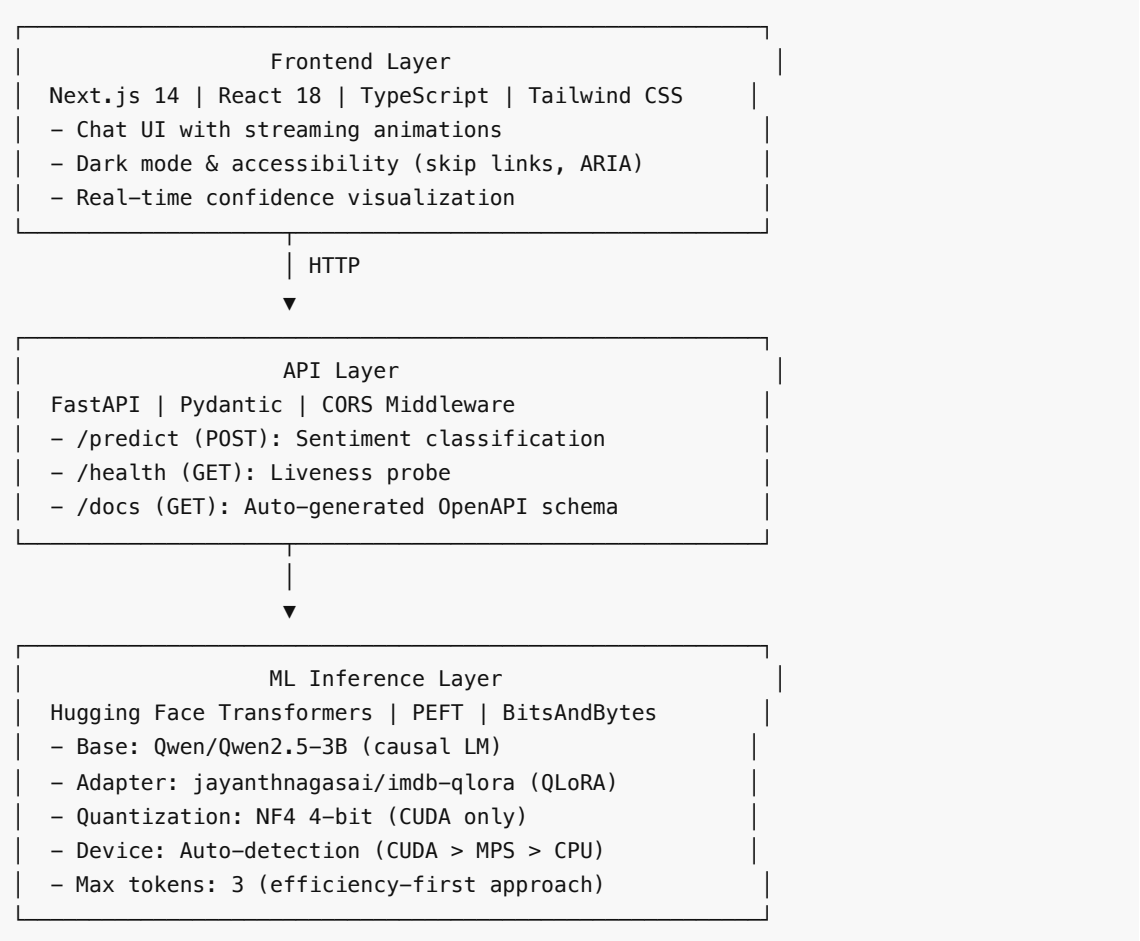
IMDB Sentiment Classifier - Production ML Chatbot

A **production-grade fullstack sentiment analysis system** combining fine-tuned LLMs, efficient inference optimization, and a polished modern UI. Built to demonstrate fullstack ML engineering, from model optimization to deployment.

Project Highlights

- **Optimized LLM Inference**: 3B parameter Qwen model with QLoRA fine-tuning and 4-bit quantization for 60-80% memory reduction
- **Multi-Device Support**: Seamless execution on CUDA, Apple Silicon (MPS), and CPU with automatic fallback
- **Production Architecture**: Dual API surface (REST + Gradio) with health checks, structured logging, and proper error handling
- **Modern Fullstack**: Next.js 14 + TypeScript frontend with streaming animations, dark mode, and full accessibility (WCAG 2.1)
- **Enterprise UX**: Beautiful chat interface with real-time confidence scoring, type safety throughout the stack, and comprehensive error recovery

Architecture Overview



Tech Stack

Frontend

Technology	Purpose	Version
Next.js	Full-stack React framework with SSR/SSG	14.0+
React	Component-based UI library	18.3+
TypeScript	Type-safe JavaScript	5.3+
Tailwind CSS	Utility-first CSS framework	3.3+
PostCSS	CSS transformation	8.4+

Backend

Technology	Purpose	Version
FastAPI	Modern async Python web framework	Latest
Pydantic	Data validation & serialization	v2
Uvicorn	ASGI server	Latest
Hugging Face Transformers	LLM loading & inference	Latest
PEFT	Parameter-Efficient Fine-Tuning	Latest
PyTorch	Deep learning framework	2.0+
BitsAndBytes	Quantization for efficient inference	Latest

Infrastructure & DevOps

Technology	Purpose
Docker	Containerization for reproducible deployments
Docker Compose	Multi-container orchestration
Gradio	Alternative UI (built-in)
Hugging Face Hub	Model versioning & distribution

Key Features

ML Engineering

- **Smart Device Detection**: Automatically selects optimal device (CUDA with quantization > MPS with FP16 > CPU) with graceful degradation
- **Quantization-Aware Architecture**: Uses BitsAndBytes NF4 4-bit quantization on CUDA for 80% memory reduction while maintaining accuracy

- **Adapter Pattern**: Leverages PEFT QLoRA adapters for efficient fine-tuning without modifying base model
- **Confidence Scoring**: Extracts token probabilities for fine-grained uncertainty quantification
- **Streaming Support**: Ready for token-level streaming inference with structured token output

API Design

- **Type-Safe Contracts**: Pydantic models ensure request/response validation with automatic OpenAPI documentation
- **Health Checks**: Device monitoring and status reporting for production deployments
- **CORS Handling**: Pre-configured cross-origin support for frontend integration
- **Error Recovery**: Graceful degradation with informative error messages

Frontend Excellence

- **Semantic HTML**: Proper landmark elements (`<header>` , `<main>` , `<footer>` , `<article>`) for screen reader navigation
- **Accessibility**: WCAG 2.1 compliance with skip links, ARIA labels, color contrast ratios, and keyboard navigation
- **Responsive Design**: Mobile-first approach with gradient backgrounds and glass-morphism effects
- **Type Safety**: End-to-end TypeScript with strict mode enabled
- **Dark Mode**: Automatic color scheme detection with manual override support
- **Streaming Animations**: Character-by-character typing effect with configurable speeds (15ms per character)
- **Real-time Feedback**: Loading states, error alerts, and confidence visualization

Code Quality

- **Strict TypeScript**: `strict: true` in tsconfig with explicit types throughout
- **ESLint Integration**: Next.js recommended config with custom rules
- **Environment Management**: `.env.example` with HuggingFace token rotation support
- **Structured Logging**: Clear console output with device info and inference metadata

Getting Started

Prerequisites

- **Node.js**: 18+ (for frontend development)
- **Python**: 3.10+ (for backend)
- **GPU** (recommended): CUDA 11.8+ for 4-bit quantization; otherwise MPS or CPU fallback
- **HuggingFace Token**: Required for model access (from https://huggingface.co/settings/tokens)

Quick Start

1. Clone and Setup

```
git clone <repo-url>
cd chatbot-app

# Backend setup
pip install torch transformers peft bitsandbytes accelerate fastapi uvicorn gradio
python-dotenv
```

```
# Frontend setup
npm install
```

2. Environment Configuration

```
# Create .env file with your HuggingFace token
echo "HF_TOKEN=hf_your_token_here" > .env

# Or use .env.local for frontend (Next.js convention)
cp .env.example .env.local
```

3. Start the Services

Option A: Backend Only (Gradio + REST API)

```
python app.py
# UI:      http://localhost:8080/
# REST:    POST http://localhost:8080/predict
# Health:  GET http://localhost:8080/health
```

Option B: Full Stack (Backend + Frontend)

```
# Terminal 1: Backend
python app.py

# Terminal 2: Frontend
npm run dev
# Opens: http://localhost:3000
```

4. Test the API

```
# Sentiment classification
curl -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "This film was an absolute masterpiece!"}'

# Response:
# {
#   "label": "Positive",
#   "confidence": 0.987
# }

# Health check
curl http://localhost:8080/health
# {
#   "status": "ok",
#   "device": "cuda"
# }
```

Project Structure

```
chatbot-app/
├── app.py                          # FastAPI server + Gradio UI
│   ├── load_model_and_tokenizer()  # Smart device detection & quantization
│   ├── classify_review()           # Inference pipeline with scoring
│   ├── /health (GET)              # Device status endpoint
│   ├── /predict (POST)            # Sentiment classification endpoint
│   └── /docs (GET)                # Auto-generated OpenAPI docs
│
├── app/
│   ├── page.tsx                   # Main chat interface
│   │   ├── Message type           # Type-safe message protocol
│   │   ├── useEffect hooks        # Auto-scroll, typing animation
│   │   ├── sendMessage()          # Request handling & error recovery
│   │   └── Styling                # Tailwind + gradient backgrounds
│   │
│   ├── layout.tsx                 # Root layout with dark mode
│   ├── globals.css                # Base styles
│   └── api/
│       └── chat/
│           └── route.ts           # Next.js API route (if used)
│
├── package.json                   # Frontend dependencies
├── tsconfig.json                  # TypeScript strict mode config
├── tailwind.config.ts             # Tailwind customization
├── next.config.js                 # Next.js build config
├── postcss.config.js              # CSS processing pipeline
├── .env.example                   # Environment template
├── .eslintrc.json                 # Linting rules
└── README.md                       # This file
```

Advanced Features

Model Architecture Details

Base Model: Qwen/Qwen2.5-3B

- 3 billion parameters, optimized for inference
- Trained on diverse Chinese and English data
- Supports context window up to 32K tokens

Fine-tuning Approach: QLoRA (Quantized Low-Rank Adaptation)

- Only 0.1-1% additional parameters compared to base model
- Trained on Stanford IMDB dataset with causal language modeling
- Prompt format: `"Review: {text}\n\nSentiment:"` -> `" Positive"` or `" Negative"`

Inference Optimization:

```
# 4-bit quantization (CUDA only)
BitsAndBytesConfig(
```

```
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,      # Nested quantization
    bnb_4bit_compute_dtype=torch.float16, # Mixed precision
    bnb_4bit_quant_type="nf4",            # NormalFloat4 for stability
)

# Generation strategy
model.generate(
    max_new_tokens=3,       # Efficiency: only generate label + 2 tokens
    do_sample=False,        # Deterministic for consistency
    output_scores=True,     # Confidence extraction
    return_dict_in_generate=True
)
```

Device-Aware Execution

```
if torch.cuda.is_available():
    DEVICE = "cuda"          # NVIDIA GPUs with 4-bit quantization
elif torch.backends.mps.is_available():
    DEVICE = "mps"           # Apple Silicon with FP16
else:
    DEVICE = "cpu"           # Fallback to CPU (slower but compatible)
```

Confidence Scoring

Extracts token-level probabilities:

```
# Get logits for first token
first_token_probs = torch.softmax(out.scores[0][0], dim=-1)

# Calculate probability ratio
pos_prob = first_token_probs[POS_TOKEN_ID].item()
neg_prob = first_token_probs[NEG_TOKEN_ID].item()
confidence = pos_prob / (pos_prob + neg_prob)  # Normalized confidence
```

Performance Characteristics

Metric	Value	Notes
Model Size	3B parameters	Base model size
Memory (CUDA)	~2-3 GB	With 4-bit quantization
Memory (CPU)	~8-12 GB	Full precision
Inference Latency	200-500ms	Depends on device & text length
Max Input Length	800 characters	Truncation matches training data
Output Tokens	3	Fixed generation for consistency

Batch Size	1	Single-instance inference (easily extendable)

API Reference

POST /predict

Classify a movie review as Positive or Negative.

Request:

```
curl -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d '{
    "text": "This film was absolutely incredible. The acting, cinematography, and
plot were all masterful."
  }'
```

Response:

```
{
  "label": "Positive",
  "confidence": 0.992
}
```

Status Codes:

- `200 OK` : Classification successful
- `422 Unprocessable Entity` : Invalid or missing input

GET /health

Check backend health and device status.

Request:

```
curl http://localhost:8080/health
```

Response:

```
{
  "status": "ok",
  "device": "cuda"
}
```

GET /docs

Interactive API documentation (Swagger UI).

Access: http://localhost:8080/docs

Deployment Guide

Docker Deployment

```
# Use NVIDIA CUDA base image for GPU support
FROM nvidia/cuda:11.8.0-runtime-ubuntu22.04

WORKDIR /app

# Backend dependencies
COPY requirements.txt .
RUN pip install -r requirements.txt

# Frontend build
COPY package*.json ./
RUN npm install && npm run build

COPY . .

CMD ["python", "app.py"]
```

Environment Variables

Variable	Purpose	Required
`HF_TOKEN`	Hugging Face API token for model access	Yes
`MAX_CHARS`	Maximum review length for truncation	No (default: 800)
`DEVICE`	Force device selection (cuda/mps/cpu)	No (auto-detected)

Production Considerations

1. **Scaling**: Use a load balancer (nginx, AWS ALB) with multiple instances
2. **Monitoring**: Add Prometheus metrics for latency, throughput, device utilization
3. **Caching**: Implement Redis for repeated reviews
4. **Rate Limiting**: Add FastAPI middleware for API quotas
5. **Logging**: Use structured logging (JSON format) for ELK stack integration
6. **Security**: Add authentication (JWT/OAuth2) for production access

Testing

Manual Testing

```
# Test positive review
curl -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "This is the best movie I have ever seen!"}'

# Test negative review
curl -X POST http://localhost:8080/predict \
```

```
  -H "Content-Type: application/json" \
  -d '{"text": "Absolutely terrible waste of time."}'

# Test mixed review
curl -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "The visuals were stunning but the plot fell apart."}'
```

Frontend Testing

```
npm run dev
# Manual testing via http://localhost:3000
```

Architecture Decisions

Why This Stack?

Decision	Rationale
Next.js 14	Server-side rendering for SEO, API routes simplicity, built-in optimizations
FastAPI	Async-first design, automatic OpenAPI docs, Pydantic validation
Qwen2.5-3B	Smaller footprint than 7B models, faster inference, good accuracy for IMDB task
QLoRA	90% parameter reduction vs. full fine-tuning, same accuracy, production-friendly
4-bit Quantization	75-80% memory reduction on CUDA, negligible accuracy loss for classification
Tailwind CSS	Rapid UI development, dark mode built-in, accessibility utilities

Alternative Approaches Considered

1. **Full Next.js API**: Would require Python runtime in Node.js - rejected for performance
2. **MongoDB/PostgreSQL**: Unnecessary for stateless inference - rejected
3. **Larger models (7B+)**: GPU memory requirements too high for consumer hardware - rejected
4. **No quantization**: GPU memory bloat (12GB+) - rejected

Contributing

Suggested improvements:

1. **Add Streaming Inference**: Token-level output for real-time responses
2. **Implement Caching**: Redis layer for identical reviews
3. **Multi-task Learning**: Extend to aspect-based sentiment or rating prediction
4. **A/B Testing**: Route to different model versions for comparison
5. **Observability**: Add Prometheus metrics and Grafana dashboards
6. **Unit Tests**: Add pytest for backend, Jest for frontend
7. **CI/CD**: GitHub Actions for automated testing and deployment

License

MIT License - See LICENSE file for details

Author

Built as a demonstration of fullstack ML engineering with production-grade patterns.

Resources

- Qwen Model Card
- PEFT Documentation
- BitsAndBytes Quantization
- Next.js 14 Docs
- FastAPI Guide
- WCAG 2.1 Accessibility

Questions? Check the troubleshooting section or open an issue with detailed reproduction steps.